Practically Groovy: The @Delegate annotation

Exploring the frontiers of duck typing in a statically typed language

Skill Level: Intermediate

Scott Davis (scott@thirstyhead.com)
Founder
ThirstyHead.com

25 Aug 2009

Scott Davis continues the discussion about Groovy metaprogramming with an in-depth look at the `@Delegate` annotation, which blurs the distinctions between data type and behavior and static and dynamic typing.

Over the past few *Practically Groovy* articles, you've seen how Groovy language features like closures and metaprogramming add new dynamic capabilities to Java™ development. This article offers more of the same. You'll see how the `@Delegate` annotation is a variation on the same `delegate` used by the `ExpandoMetaClass`. You'll also see (once again) how Groovy's dynamic capabilities make it an ideal language for unit testing.

In "Metaprogramming with closures, `ExpandoMetaClass`, and categories," you were introduced to the notion of a `delegate`. When adding a `shout()` method to the `ExpandoMetaClass` of `java.lang.String`, you used `delegate` to express the relationship between the two classes, as shown in Listing 1:

Listing 1. Using delegate to access String.toUpperCase()

```
String.metaClass.shout = {->
  return delegate.toUpperCase()
}

println "Hello MetaProgramming".shout()

//output
```

```
HELLO METAPROGRAMMING
```

You can't say `this.toUpperCase()`, because `ExpandoMetaClass` doesn't have a `toUpperCase()` method. Similarly, you can't say `super.toUpperCase()`, because `ExpandoMetaClass` doesn't extend `String`. (In fact, it couldn't possibly extend `String`, because `String` is a `final` class.) The Java language simply doesn't have the vocabulary to express the symbiotic relationship between the two classes. This is why Groovy introduced the concept of a `delegate`.

About this series

Groovy is a modern programming language that runs on the Java platform. It offers seamless integration with existing Java code while introducing dramatic new features like closures and metaprogramming. Put simply, Groovy is what the Java language would look like had it been written in the 21st century.

The key to incorporating any new tool into your development toolkit is knowing when to use it and when to leave it in the box. Groovy can be extremely powerful, but only when applied properly to appropriate scenarios. To that end, the *Practically Groovy* series explores the practical uses of Groovy, helping you learn when and how to apply them successfully.

In Groovy 1.6, a `@Delegate` annotation was added to the language. (See Resources for a list of all of the new annotations added to Groovy 1.6.) This annotation allows you to add one or more delegates to *any* of your classes — not just `ExpandoMetaClass`.

To gain a full appreciation of the `@Delegate` annotation's power, consider a common but difficult proposition in Java programming: creating a new class based on a `final` class.

The composite pattern and final classes

Suppose that you would like to create an `AllCapsString` class that has all of the behavior of `java.lang.String`, except — as the name implies — the value is always returned in all uppercase. `String` is a `final` class — the Java equivalent of an evolutionary dead end. Listing 2 proves that you cannot extend `String` directly:

Listing 2. Impossibility of extending a final class

```
class AllCapsString extends String{
}

$ groovyc AllCapsString.groovy

org.codehaus.groovy.control.MultipleCompilationErrorsException:
startup failed, AllCapsString.groovy: 1: You are not allowed to
```

```
overwrite the final class 'java.lang.String'.
 @ line 1, column 1.
   class AllCapsString extends String{
   ^

1 error
```

That didn't work, so your next-best option is to use the composite pattern, as shown in Listing 3 (for more on the composite pattern, see Resources):

Listing 3. Using composition for a new type of String class

```
class AllCapsString{
  final String body

  AllCapsString(String body){
    this.body = body.toUpperCase()
  }

  String toString(){
    body
  }

  //now implement all 72 String methods
  char charAt(int index){
    return body.charAt(index)
  }

  //snip...
  //one method down, 71 more to go...
}
```

So, the `AllCapsString` class *has* a `String`, but it won't *behave* like a `String` unless you mirror all 72 `String` methods. To see the methods you need to add, either refer to the Javadocs for `String` or run the code in Listing 4:

Listing 4. Printing out all of the methods for the String class

```
String.class.methods.eachWithIndex{method, i->
  println "${i} ${method}"
}

//output
0 public boolean java.lang.String.contentEquals(java.lang.CharSequence)
1 public boolean java.lang.String.contentEquals(java.lang.StringBuffer)
2 public boolean java.lang.String.contains(java.lang.CharSequence)
...
```

Adding 72 `String` methods to `AllCapsString` by hand is not a wise use of your precious developer time. This is where the `@Delegate` annotation comes in handy.

Understanding @Delegate

`@Delegate` is a compile-time annotation that instructs the compiler to push all of the

delegate's methods and interfaces up to the outer class.

Before you add the `@Delegate` annotation to `body`, compile `AllCapsString` and use `javap` to verify that most of the `String` methods are indeed missing, as shown in Listing 5:

Listing 5. AllCapsString, before using @Delegate

```
$ groovyc AllCapsString.groovy
$ javap AllCapsString
Compiled from "AllCapsString.groovy"
public class AllCapsString extends java.lang.Object
       implements groovy.lang.GroovyObject{
    public AllCapsString(java.lang.String);
    public java.lang.String toString();
    public final java.lang.String getBody();
    //snip...
```

Now, add the `@Delegate` annotation to `body` as shown in Listing 6. Repeat the `groovyc` and `javap` commands and voila, `AllCapsString` has all of the same methods and interfaces as `java.lang.String`.

Listing 6. Using the @Delegate annotation to push all of the String methods up to the surrounding class

```
class AllCapsString{
  @Delegate final String body

  AllCapsString(String body){
    this.body = body.toUpperCase()
  }

  String toString(){
    body
  }
}


$ groovyc AllCapsString.groovy
$ javap AllCapsString
Compiled from "AllCapsString.groovy"
public class AllCapsString extends java.lang.Object
       implements java.lang.CharSequence, java.lang.Comparable,
       java.io.Serializable,groovy.lang.GroovyObject{

    //NOTE: AllCapsString methods:
    public AllCapsString(java.lang.String);
    public java.lang.String toString();
    public final java.lang.String getBody();

    //NOTE: java.lang.String methods:
    public boolean contains(java.lang.CharSequence);
    public int compareTo(java.lang.Object);
    public java.lang.String toUpperCase();
    //snip...
```

Notice, however, that you can still call `getBody()` and therefore bypass all of the methods pushed up to the surrounding `AllCapsString` class. By adding `private` to the field declaration — `@Delegate final private String body` — you can suppress the normal getter/setter methods from appearing. This completes the transformation: `AllCapsString` provides the full behavior of a `String`, allowing you to override native `String` methods as appropriate.

The caveats of duck typing in a static language

Although `AllCapsString` now has all of the behavior of a `String`, it still isn't *truly* a `String`. In Java code, you can't use `AllCapsString` as a drop-in replacement for `String`, because it isn't truly a duck — it only quacks like one. (Dynamic languages are said to use *duck* typing; the Java language uses *static* typing. See Resources for more on the difference.) In other words, because `AllCapsString` doesn't truly extend `String` (or implement the nonexistent `Stringable` interface), you can't use it interchangeably with `String` in Java code. Listing 7 shows an example of how casting an `AllCapsString` to a `String` fails in the Java language:

Listing 7. Static typing in the Java language preventing AllCapsString from being used interchangeably with String

```
public class JavaExample{
  public static void main(String[] args){
    String s = new AllCapsString("Hello");
  }
}

$ javac JavaExample.java
JavaExample.java:5: incompatible types
found   : AllCapsString
required: java.lang.String
    String s = new AllCapsString("Hello");
               ^
1 error
```

So, although Groovy's `@Delegate` doesn't actually subvert Java's `final` keyword by allowing you to extend a class that the original developer has explicitly blocked from being extended, it gives you as much power as you can get without crossing the line.

And remember that your class can have more than one `delegate`. Suppose that you would like to create a `RemoteFile` class that shares the characteristics of both a `java.io.File` and a `java.net.URL`. The Java language doesn't support multiple inheritance, but you can come awfully close with a couple of `@Delegates`, as shown in Listing 8. The `RemoteFile` class is neither a `File` or a `URL`, but it has the behavior of both.

Listing 8. Multiple @Delegates, offering the behavior of multiple inheritance

```
class RemoteFile{
  @Delegate File file
  @Delegate URL url
}
```

If the `@Delegate` can change only the behavior of your class — not the type — does that mean that it is worthless to Java developers? Hardly. Even statically typed languages like the Java language offer a limited form of duck typing called *polymorphism*.

Quacking like a polymorphic duck

Polymorphism — a word derived from the Greek for "many shapes" — means that as long as a group of classes explicitly share the same behavior by implementing the same interface, they can be used interchangeably. In other words, if you define a variable of `Duck` type (assuming that `Duck` is an interface that formally defines the `quack()` and `waddle()` methods), you can assign a `new Mallard()` to it, a `new GreenWingedTeal()`, or (my favorite) a `new PekingWithHoisinSauce()`.

The `@Delegate` annotation fully supports polymorphism by promoting not only the delegate class's methods to the outer class, but the delegate's interfaces as well. This means that if the delegate class implements an interface, you are back in the business of creating a drop-in replacement for it.

@Delegate and the List interface

Suppose that you would like to create a new class called `FixedList`. It should behave just like a `java.util.ArrayList` with one important distinction: you should be able to define an upper limit to the number of elements that you can add to it. This allows you to create a `sportsCar` variable that can seat up to two passengers but no more, a `restaurantTable` that can seat up to four diners but no more, and so on.

The `ArrayList` class implements the `List` interface. This gives you a couple of options. You could have your `FixedList` class implement the `List` interface as well, but then you are faced with the unsavory task of providing an implementation for all of the `List` methods. Because `ArrayList` isn't a `final` class, another option would be simply to have `FixedList` extend `ArrayList`. This is a perfectly valid course of action, but if (hypothetically) `ArrayList` were declared `final`, the `@Delegate` annotation provides a third option: by making an `ArrayList` a delegate of `FixedList`, you could get all of the behavior of an `ArrayList` while automatically implementing the `List` interface as well.

To begin, create the `FixedList` class with an `ArrayList` delegate, as shown in Listing 9. Do the `groovyc` / `javap` dance to verify that `FixedList` offers not only the same methods as an `ArrayList`, but also the same interfaces.

Listing 9. First pass at creating the FixedList class

```
class FixedList{
  @Delegate private List list = new ArrayList()
  final int sizeLimit

  /**
    * NOTE: This constructor limits the max size of the list,
    *  not just the initial capacity like an ArrayList.
    */
  FixedList(int sizeLimit){
    this.sizeLimit = sizeLimit
  }
}

$ groovyc FixedList.groovy
$ javap FixedList
Compiled from "FixedList.groovy"
public class FixedList extends java.lang.Object
             implements java.util.List,java.lang.Iterable,
             java.util.Collection,groovy.lang.GroovyObject{
    public FixedList(int);
    public java.lang.Object[] toArray(java.lang.Object[]);
    //snip..
```

You haven't done anything yet to constrain the size of the `FixedList`, but this is a good start. How can you verify that the size of a `FixedList` isn't, well, *fixed* at this point? You could write some throwaway sample code, but if `FixedList` is going into production, you'd be better served by immediately writing some test cases for it.

Testing @Delegate with GroovyTestCase

To begin testing `@Delegate`, write a unit test that proves that you can add more elements to a `FixedList` than you should be able to. Listing 10 shows such a test:

Listing 10. Writing a failing test first

```
class FixedListTest extends GroovyTestCase{

  void testAdd(){
    List threeStooges = new FixedList(3)
    threeStooges.add("Moe")
    threeStooges.add("Larry")
    threeStooges.add("Curly")
    threeStooges.add("Shemp")
    assertEquals threeStooges.sizeLimit, threeStooges.size()
  }
}

$ groovy FixedListTest.groovy

There was 1 failure:
```

```
1) testAdd(FixedListTest)junit.framework.AssertionFailedError:
   expected:<3> but was:<4>
```

It looks like the `add()` method is something you should override in `FixedList`, as in Listing 11. Rerunning the test fails again, but this time because of the exception being thrown.

Listing 11. Overriding ArrayList's add() method

```
class FixedList{
  @Delegate private List list = new ArrayList()
  final int sizeLimit

  //snip...

  boolean add(Object element){
    if(list.size() < sizeLimit){
      return list.add(element)
    }else{
      throw new UnsupportedOperationException("Error adding ${element}:" +
                  " the size of this FixedList is limited to ${sizeLimit}.")
    }
  }
}

$ groovy FixedListTest.groovy

There was 1 error:
1) testAdd(FixedListTest)java.lang.UnsupportedOperationException:
   Error adding Shemp: the size of this FixedList is limited to 3.
```

Thanks to `GroovyTestCase`'s convenient `shouldFail` method, you can trap for the expected exception, as shown in Listing 12, and celebrate your first passing test:

Listing 12. The shouldFail() method catching the expected exception

```
class FixedListTest extends GroovyTestCase{
  void testAdd(){
    List threeStooges = new FixedList(3)
    threeStooges.add("Moe")
    threeStooges.add("Larry")
    threeStooges.add("Curly")
    assertEquals threeStooges.sizeLimit, threeStooges.size()
    shouldFail(java.lang.UnsupportedOperationException){
      threeStooges.add("Shemp")
    }
  }
}
```

Testing operator overloading

In "Smooth Operators," you learned that Groovy supports *operator overloading*. With `Lists`, you can use << to add elements as well as the traditional `add()` method. Write a quick unit test like the one in Listing 13 to verify that using << doesn't break

the `FixedList` unexpectedly:

Listing 13. Testing operator overloading

```
class FixedListTest extends GroovyTestCase{

  void testOperatorOverloading(){
    List oneList = new FixedList(1)
    oneList << "one"
    shouldFail(java.lang.UnsupportedOperationException){
      oneList << "two"
    }
  }
}
```

Seeing the test pass should give you some peace of mind.

You can also test pathological cases. Listing 14, for example, tests what happens when you create a `FixedList` with a negative number of elements:

Listing 14. Testing edge cases

```
class FixedListTest extends GroovyTestCase{
  void testNegativeSize(){
    List badList = new FixedList(-1)
    shouldFail(java.lang.UnsupportedOperationException){
      badList << "will this work?"
    }
  }
}
```

Testing insertion of an element in the middle of the list

Now that you are convinced that the simple overridden `add()` method works, the next step is to implement the overloaded `add()` method that takes an index as well as an element, as shown in Listing 15:

Listing 15. Adding elements with an index

```
class FixedList{
  @Delegate private List list = new ArrayList()
  final int sizeLimit

  void add(int index, Object element){
    list.add(index, element)
    trimToSize()
  }

  private void trimToSize(){
    if(list.size() > sizeLimit){
      (sizeLimit..<list.size()).each{
        list.pop()
      }
```

```
    }
  }
}
```

Notice that you can (and should) defer to the delegate's native functionality whenever possible — after all, that's why you chose it as a delegate in the first place. In this case, you let the `ArrayList` do the adding, and simply lop off any elements that exceed the size of the `FixedList`. (Whether or not this `add()` method should throw an `UnsupportedOperationException` like the other `add()` method does is a design decision you can make on your own.)

The `trimToSize()` method contains a bit of Groovy syntactic sugar worth noting. First of all, the `pop()` method is something Groovy metaprograms onto all `Lists`. It removes the last element in the `List`, last-in first-out (LIFO) style.

Next, notice the use of a Groovy `range` in the `each` loop. Replacing variables with real numbers might help clarify the behavior. Suppose that the `FixedList` has a `sizeLimit` of 3, and after new elements are added it has a `size()` of 5. The range would then look like `(3..5).each{}`. But because of the base-0 notation used for `Lists`, no element in the list has an index of 5. By saying `(3..<5).each{}`, you exclude the final number in the range.

Write a couple of tests, as in Listing 16, to verify that the new overloaded `add()` method behaves as expected:

Listing 16. Testing adding elements to the middle of the FixedList

```
class FixedListTest extends GroovyTestCase{
  void testAddWithIndex(){
    List threeStooges = new FixedList(3)
    threeStooges.add("Moe")
    threeStooges.add("Larry")
    threeStooges.add("Curly")
    threeStooges.add(2,"Shemp")
    assertEquals 3, threeStooges.size()
    assertFalse threeStooges.contains("Curly")
  }


  void testAddWithIndexOnALessThanFullList(){
    List threeStooges = new FixedList(3)
    threeStooges.add("Curly")
    assertEquals 1, threeStooges.size()

    threeStooges.add(0, "Larry")
    assertEquals 2, threeStooges.size()
    assertEquals "Larry", threeStooges[0]

    threeStooges.add(0, "Moe")
    assertEquals 3, threeStooges.size()
    assertEquals "Moe", threeStooges[0]
    assertEquals "Larry", threeStooges[1]
    assertEquals "Curly", threeStooges[2]
  }
}
```

Did you notice that you wrote more test code than production code? Good! I like saying that you should have at least two pounds of test code for every pound of production code.

Implementing the addAll() methods

To complete the `FixedList` class, override the `addAll()` methods in `ArrayList`, as shown in Listing 17:

Listing 17. Implementing the addAll() methods

```
class FixedList{
  @Delegate private List list = new ArrayList()
  final int sizeLimit

  boolean addAll(Collection collection){
    def returnValue = list.addAll(collection)
    trimToSize()
    return returnValue
  }

  boolean addAll(int index, Collection collection){
    def returnValue = list.addAll(index, collection)
    trimToSize()
    return returnValue
  }
}
```

Now write the corresponding unit tests, shown in Listing 18:

Listing 18. Testing the addAll() methods

```
class FixedListTest extends GroovyTestCase{
  void testAddAll(){
    def quartet = ["John", "Paul", "George", "Ringo"]
    def trio = new FixedList(3)
    trio.addAll(quartet)
    assertEquals 3, trio.size()
    assertFalse trio.contains("Ringo")
  }

  void testAddAllWithIndex(){
    def quartet = new FixedList(4)
    quartet << "John"
    quartet << "Ringo"
    quartet.addAll(1, ["Paul", "George"])
    assertEquals "John", quartet[0]
    assertEquals "Paul", quartet[1]
    assertEquals "George", quartet[2]
    assertEquals "Ringo", quartet[3]
  }
}
```

And there you have it. Thanks to the power of the `@Delegate` annotation, you were able to create a `FixedList` class in about 50 lines of code. Thanks to the power of `GroovyTestCase`, you were able to test it, thereby allowing you to put this code into production with confidence that it behaves as expected. Listing 19 shows the `FixedList` class in its entirety:

Listing 19. The complete FixedList class

```
class FixedList{
  @Delegate private List list = new ArrayList()
  final int sizeLimit

  /**
    * NOTE: This constructor limits the max size of the list,
    *  not just the initial capacity like an ArrayList.
    */
  FixedList(int sizeLimit){
    this.sizeLimit = sizeLimit
  }

  boolean add(Object element){
    if(list.size() < sizeLimit){
      return list.add(element)
    }else{
      throw new UnsupportedOperationException("Error adding ${element}:" +
        " the size of this FixedList is limited to ${sizeLimit}.")
    }
  }

  void add(int index, Object element){
    list.add(index, element)
    trimToSize()
  }

  private void trimToSize(){
    if(list.size() > sizeLimit){
      (sizeLimit..<list.size()).each{
        list.pop()
      }
    }
  }

  boolean addAll(Collection collection){
    def returnValue = list.addAll(collection)
    trimToSize()
    return returnValue
  }

  boolean addAll(int index, Collection collection){
    def returnValue = list.addAll(index, collection)
    trimToSize()
    return returnValue
  }

  String toString(){
    return "FixedList size: ${sizeLimit}\n" + "${list}"
  }
}
```

Conclusion

By focusing on the ability to add new *behavior* to classes instead of transforming their *type*, Groovy's metaprogramming capabilities open up a whole new set of dynamic possibilities without violating the rules of the Java language's static type system. Between the `ExpandoMetaClass` (which lets you add arbitrary new methods to existing classes by wrapping them) and `@Delegate` (which lets you expose the functionality of composite, interior classes through the outer wrapping class), Groovy allows the JVM to waddle and quack like never before.

In the next article, I'll demonstrate an old technology that is garnering fresh, new interest thanks to Groovy's flexible syntax: Swing. Yes, the complexity of Swing melts away with Groovy's `SwingBuilder`. It makes desktop development — dare I say it? — fun and easy. Until then, I hope that you find plenty of practical uses for Groovy.

Downloads

Description	Name	Size	Download method
Source code for this article's examples	j-pg08259.zip	5KB	HTTP

Information about download methods

Resources

Learn

- Groovy: Learn more about Groovy at the project Web site.
- "What's New in Groovy 1.6" (Guillaume LaForge, InfoQ, February 2009): Read about new annotations and other features in the latest Groovy release.
- Composite pattern: Check out this Wikipedia entry on the composite design pattern.
- Duck typing: Learn more about duck typing in this Wikipedia article.
- "*Crossing borders*: Typing strategies beyond the Java model" (Bruce Tate, developerWorks, May 2006): Read an in-depth discussion of typing strategies.
- AboutGroovy.com: Keep up with the latest Groovy news and article links.
- *Groovy Recipes* (Scott Davis, Pragmatic Programmers, 2008): Learn more about Groovy and Grails in Scott Davis' latest book.
- *Mastering Grails*: Scott Davis's companion series focuses on this Groovy-based platform for Web development.
- Browse the technology bookstore for books on these and other technical topics.
- developerWorks Java technology zone: Find hundreds of articles about every aspect of Java programming.

Get products and technologies

- Groovy: Download the latest Groovy ZIP file or tarball.

Discuss

- Groovy mailing list: Browse, search, or subscribe to the Groovy mailing list.
- Get involved in the My developerWorks community.

About the author

Scott Davis
Scott Davis is an internationally recognized author, speaker, and software developer. He is the founder of ThirstyHead.com, a Groovy and Grails training company. His books include *Groovy Recipes: Greasing the Wheels of Java*, *GIS for Web Developers: Adding Where to Your Application*, *The Google Maps API*, and *JBoss At Work*. He writes two ongoing article series for IBM developerWorks: *Mastering Grails* and *Practically Groovy*.

Trademarks

Java and all Java-based trademarks are trademarks of Sun Microsystems, Inc. in the United States, other countries, or both.